

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

Jamal Tannous 208912337

Dan Ydov 039992391

Basel Mansour 208204859

1. Objective.....	2
2. Test Environment Setup.....	2
2.1 Implementation Environment.....	2
2.2 Default Simulation Parameters.....	3
2.3 Scenario Configuration.....	4
3. Agent Strategy Under Test.....	5
4. Strategic Comparison: Baseline vs. Advanced Strategy Behavior.....	6
5. Evaluation Metrics.....	8
5.1 System Metrics.....	8
5.2 Utility Function.....	9
5.3 Social Welfare.....	9
6. Results Summary.....	10
7. Data Visualization.....	11
7.1 Throughput by Scenario.....	11
7.2 Average Completion Time by Scenario.....	12
7.3 Social Welfare by Scenario.....	13
7.4 Average Utility per Agent.....	14
7.5 Invalid Moves Prevented.....	15
7.6 Deadlocks Detected.....	16
8. Stress Testing.....	17
8.1 High-Load Scenario.....	17
8.2 High-Density Scenario.....	18
8.3 Deadlock Scenario.....	19
8.4 Long High-Load Run: 6000 Timesteps.....	20
9. Discussion of Results.....	21
9.1 Why Throughput Increased Under Load.....	21
9.2 Why High Density Reduced Average Utility.....	21
9.3 Why Collision Events Stayed at Zero.....	21
9.4 Why Deadlocks Appeared but Did Not Break the Simulation.....	22
9.5 Best Overall Scenario.....	22
10. Threats to Validity and Limitations.....	23
10.1 Mapping Results to SRS/SDD Requirements.....	23
11. Conclusion.....	25

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

1. Objective

The objective of this validation report is to evaluate whether the implemented multi-agent warehouse strategy achieves its intended goals through simulation, metric collection, data analysis, and stress testing.

The tested system simulates warehouse robots that dynamically receive pickup/drop-off tasks, bid for tasks, plan paths, avoid collisions, recover from blocking and deadlock situations, and export performance evidence through logs and metrics.

The main validation questions are:

1. Can the agents complete pickup/drop-off tasks successfully?
2. Does the auction protocol assign tasks deterministically and without duplicated ownership?
3. Does Cooperative A* with a reservation table prevent collisions?
4. Does the system remain functional under high load and high agent density?
5. Does the engineered strategy produce acceptable utility and social welfare under different operating conditions?

2. Test Environment Setup

2.1 Implementation Environment

The simulator was implemented in Python using a synchronous discrete-time simulation loop.

Component	Value
Language	Python 3.x
Visualization	Pygame
Simulation type	Discrete timestep simulation
Main run script	run_simulation.py
Scenario runner	run_all_scenarios.py
Output formats	JSON and CSV

The graphical interface is used for visualization and demonstration. The JSON and CSV files are used as the main validation evidence.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

2.2 Default Simulation Parameters

Parameter	Value
Grid rows	16
Grid columns	24
Default number of agents	6
Baseline task generation rate	0.18
Default random seed	7
Agent speed	1 cell per timestep
Reservation horizon	18 timesteps
Wait threshold	5 timesteps
Baseline run length	300 timesteps

The default warehouse contains fixed obstacles, pickup stations, and drop-off stations. The layout is deterministic so that repeated runs are reproducible.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

2.3 Scenario Configuration

The following scenarios were used for validation:

Scenario	Agents	Task generation rate	Purpose
baseline	6	0.18	Normal operating condition
high_load	8	0.45	Frequent task generation
high_density	14	0.22	Congested multi-agent condition
deadlock	4	0.35	Narrow-corridor deadlock stress test
dynamic	8	0.25	Medium-load dynamic environment
scalability	12	0.30	Larger-agent-count scalability test

A long high-load stress run was also executed for 6000 timesteps to evaluate stability over extended operation.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

3. Agent Strategy Under Test

The implemented strategy is an engineered multi-agent coordination stack composed of the following components:

1. Auction-based task allocation
Idle agents compute bids for available tasks. The lowest bid wins. Ties are resolved by the lowest agent ID.
2. Cooperative A* path planning
Agents plan paths in space-time rather than only in grid space.
3. Reservation-table collision prevention
The reservation table prevents two agents from occupying the same cell at the same timestep and prevents edge-swap conflicts.
4. Conflict prevention and replanning
Unsafe moves are blocked before execution. Agents wait or replan when needed.
5. Deadlock detection and recovery
Persistent blocking or waiting patterns are detected and resolved through deterministic recovery and replanning.

This strategy is designed to maximize task completion while preventing collisions and maintaining coordinated behavior under congestion.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

4. Strategic Comparison: Baseline vs. Advanced Strategy Behavior

This validation compares two implemented strategy levels: a naive baseline strategy and the advanced engineered strategy.

The naive baseline strategy represents a simple greedy multi-agent approach. In this strategy, each new task is assigned to the nearest idle agent using distance only. Agents use standard A* path planning and wait when blocked. This baseline does not use auction bidding, congestion-aware bidding, Cooperative A*, time-space reservations, edge-swap prevention, or deadlock recovery.

The advanced engineered strategy uses deterministic auction-based task allocation, Cooperative A* path planning, reservation-table collision prevention, edge-swap prevention, congestion-aware bidding, replanning, and deadlock detection/recovery.

The purpose of this comparison is to show whether the engineered strategy provides measurable improvement over a simpler baseline controller. A naive strategy may be acceptable in sparse conditions, but it is expected to degrade under congestion because agents compete for the same corridors, pickup

Strategy	Task allocation	Path planning	Collision prevention	Deadlock handling	Congestion awareness
Naive baseline	Nearest idle agent	Standard A*	Current occupancy check only	None	No
Advanced strategy	Deterministic auction bidding	Cooperative A*	Reservation table and edge-swap prevention	Persistent detection and recovery	Yes

The naive baseline and the advanced strategy were both tested on the baseline warehouse scenario for 300 timesteps. The results show a clear difference in safety, productivity, coordination quality, and utility.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

Strategy	Completed tasks	Throughput	Avg completion time	Collision events	Invalid moves prevented	Utilization	Social welfare	Avg utility
Naive baseline	12	0.040	40.83	1	1194	0.263	-57.80	-9.63
Advanced strategy	42	0.140	44.10	0	58	0.838	387.35	64.56

The advanced strategy completed 42 tasks compared with only 12 tasks for the naive baseline. This is a 3.5× improvement in completed tasks. Throughput also improved from 0.040 tasks per timestep to 0.140 tasks per timestep.

The most important safety difference is collision behavior. The naive baseline produced 1 actual collision event, while the advanced strategy produced 0 actual collision events. This shows that standard A* with only current occupancy checks is not sufficient for safe multi-agent coordination. The reservation table and edge-swap prevention in the advanced strategy are necessary for reliable collision avoidance.

The naive baseline also produced 1194 invalid move preventions, compared with only 58 for the advanced strategy. This means the naive agents frequently attempted unsafe or blocked moves because they did not coordinate future movement. The advanced strategy avoided most of these failed movement attempts through Cooperative A* and time-space reservations.

The utility and social welfare results also strongly favor the advanced strategy. The naive baseline produced negative social welfare, -57.80, because low task completion and high coordination costs outweighed productive work. The advanced strategy produced positive social welfare, 387.35, and much higher average utility per agent.

Overall, the numeric comparison confirms that the engineered coordination strategy is not only more productive, but also safer and more socially beneficial than the naive baseline.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

5. Evaluation Metrics

5.1 System Metrics

The following system-level metrics were collected:

Metric	Meaning
tasks_created	Number of generated pickup/drop-off tasks
tasks_completed	Number of completed tasks
throughput	Completed tasks per simulation timestep
avg_completion_time	Average timesteps from task creation to completion
collision_events	Actual post-step collisions
invalid_moves_prevented	Unsafe moves blocked before execution
deadlocks_detected	Persistent blocking cycles detected
agent_utilization	Fraction of agent-time spent active

Throughput is defined as:

$\text{throughput} = \text{completed_tasks} / \text{total_time}$

For example, if 42 tasks are completed in 300 timesteps:

$\text{throughput} = 42 / 300 = 0.14$ tasks per timestep

This means the system completed approximately 14 tasks per 100 simulation timesteps.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

5.2 Utility Function

The assignment requires utility and social welfare analysis. Since the simulator exports task completion and coordination-cost metrics, utility was defined from those values.

For each agent i :

$$U_i = 10 \times \text{completed_tasks}_i - 0.05 \times \text{waiting_time}_i - 0.10 \times \text{replanning_count}_i \\ - 0.05 \times \text{conflicts_involved}_i - 1.00 \times \text{deadlocks_involved}_i$$

Term	Interpretation
completed_tasks_i	Positive reward for productive work
waiting_time_i	Penalty for time lost waiting
replanning_count_i	Penalty for planning instability
conflicts_involved_i	Penalty for local coordination pressure
deadlocks_involved_i	Larger penalty for severe coordination failure

The weights intentionally make task completion the dominant objective while still penalizing coordination overhead. Movement distance is not directly penalized because movement is required for productive task execution.

The reward value of 10 for task completion was chosen so that productive task execution dominates the utility score. Smaller penalties were assigned to waiting, replanning, and conflicts because these are expected coordination costs in dense multi-agent systems. Deadlock involvement receives a larger penalty because it represents a more severe coordination failure than ordinary waiting or replanning. This makes the utility function aligned with the system goal: complete as many tasks as possible while minimizing coordination overhead and severe blocking events.

5.3 Social Welfare

Social welfare is defined as the sum of all individual agent utilities:

$$\text{Social Welfare} = \sum U_i$$

This measures total system-level benefit after accounting for both completed work and coordination costs.

Average utility is also reported:

$$\text{Average Utility} = \text{Social Welfare} / \text{Number of Agents}$$

This makes scenarios with different numbers of agents easier to compare.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

6. Results Summary

Scenario	Agents	Created	Completed	Throughput	Avg completion	Collisions	Invalid moves prevented	Deadlocks	Utilization	Social welfare	Avg utility
baseline	6	58	42	0.140	44.10	0	58	4	0.838	387.35	64.56
high_load	8	128	49	0.163	85.86	0	194	16	0.802	389.80	48.73
high_density	14	73	57	0.190	47.61	0	865	16	0.457	273.75	19.55
deadlock	4	99	42	0.140	90.69	0	36	8	0.912	382.90	95.73
dynamic	8	81	56	0.187	59.12	0	75	9	0.864	506.70	63.34
scalability	12	96	77	0.257	41.10	0	308	22	0.738	620.60	51.72

The key safety result is that all tested scenarios produced zero actual collision events. This means the reservation table, conflict checking, and final movement validation successfully prevented unsafe movement before execution.

In addition to the scenario results, a direct strategy comparison was performed between the naive baseline controller and the advanced engineered controller. The advanced strategy completed 42 tasks compared with 12 for the naive baseline, achieved zero collisions compared with one collision in the naive baseline, and improved social welfare from -57.80 to 387.35. This confirms that the advanced coordination mechanisms provide measurable improvement over a simple greedy strategy.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

7. Data Visualization

7.1 Throughput by Scenario

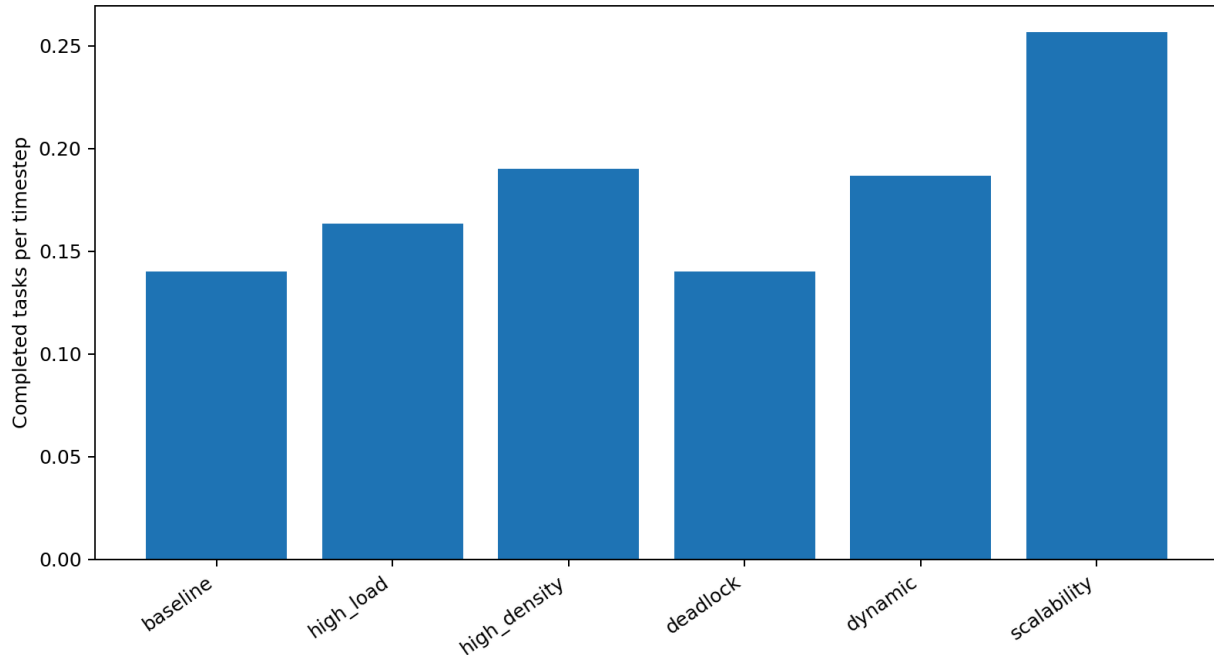


Figure 1: Throughput by scenario.

The scalability scenario achieved the highest throughput because it used more agents while still maintaining sufficient warehouse capacity. High-density and dynamic scenarios also improved throughput over baseline, but the interpretation is different: high-density increased throughput while also generating significant congestion costs.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

7.2 Average Completion Time by Scenario

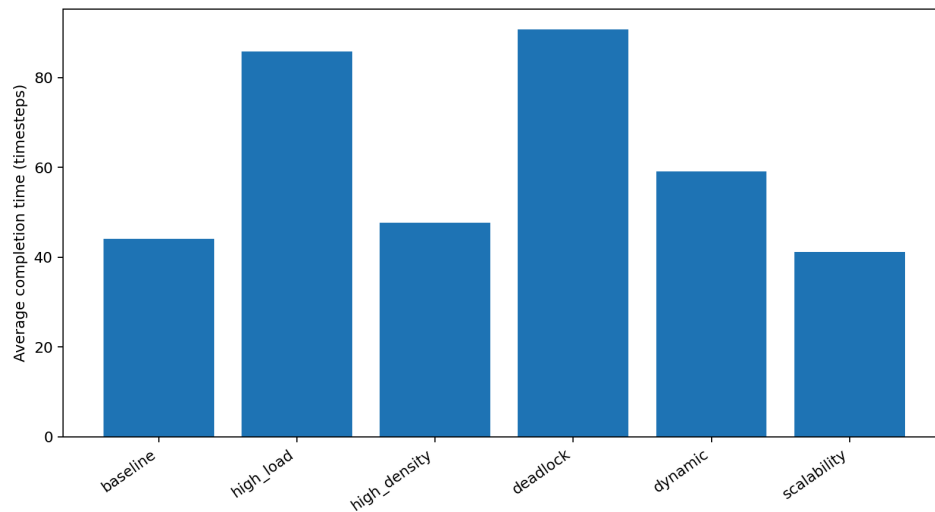


Figure 2: Average completion time by scenario.

Average completion time increased significantly in the high-load and deadlock scenarios. This is expected: high-load creates more queued tasks, and the deadlock scenario intentionally restricts movement through narrow corridors.

The scalability scenario produced the best completion-time result among the tested scenarios, showing that additional agents can improve completion time when the grid still has enough space to support them.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

7.3 Social Welfare by Scenario

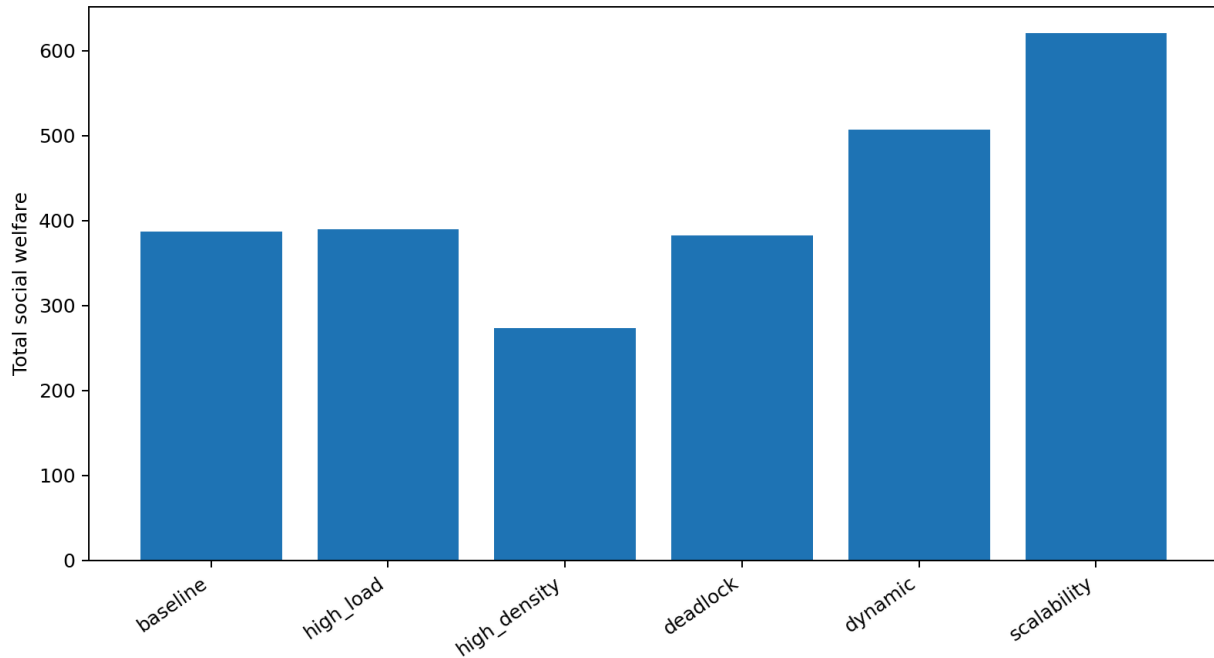


Figure 3: Social welfare by scenario.

The scalability scenario produced the highest total social welfare because it completed the most tasks. However, its average utility per agent was lower than baseline because the additional agents also introduced more waiting, replanning, and coordination costs.

The dynamic scenario produced strong social welfare with moderate coordination overhead, making it one of the most balanced scenarios.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

7.4 Average Utility per Agent

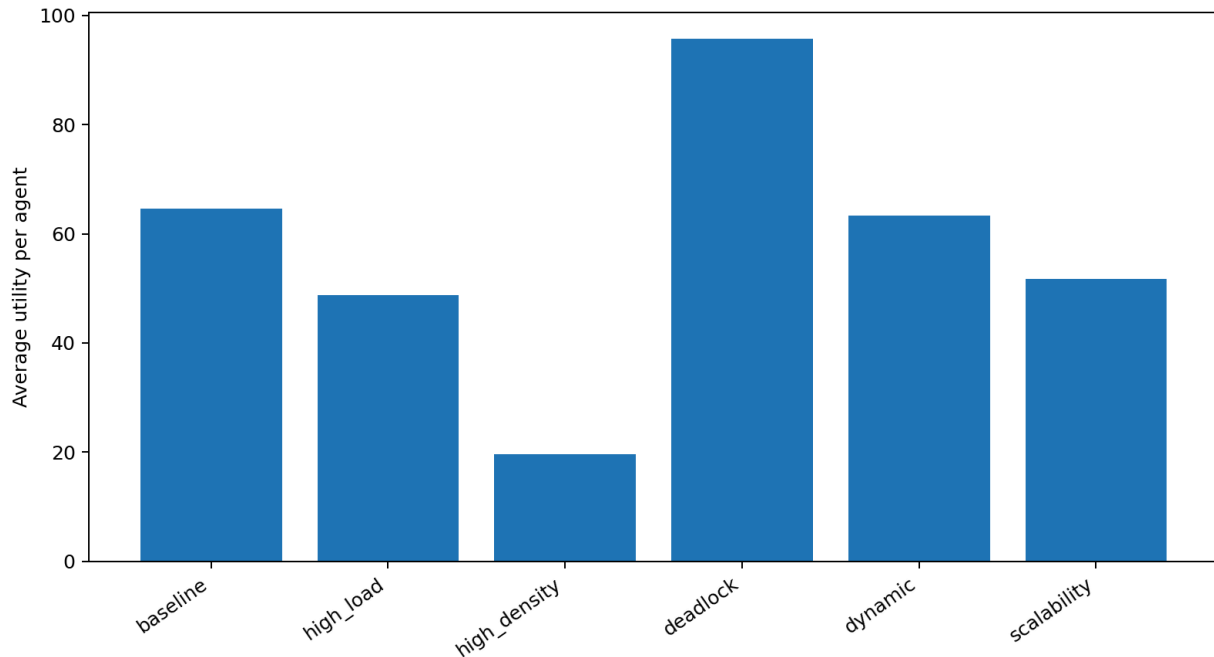


Figure 4: Average utility per agent by scenario.

The deadlock scenario has a high average utility per agent because it uses only four agents and still completes 42 tasks. However, this does not mean it is operationally superior: its average completion time is high, and the scenario is structurally narrow and artificial.

High-density has the lowest average utility per agent. This is the clearest evidence that adding too many agents into the same grid can reduce individual effectiveness even when the system completes many tasks overall.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

7.5 Invalid Moves Prevented

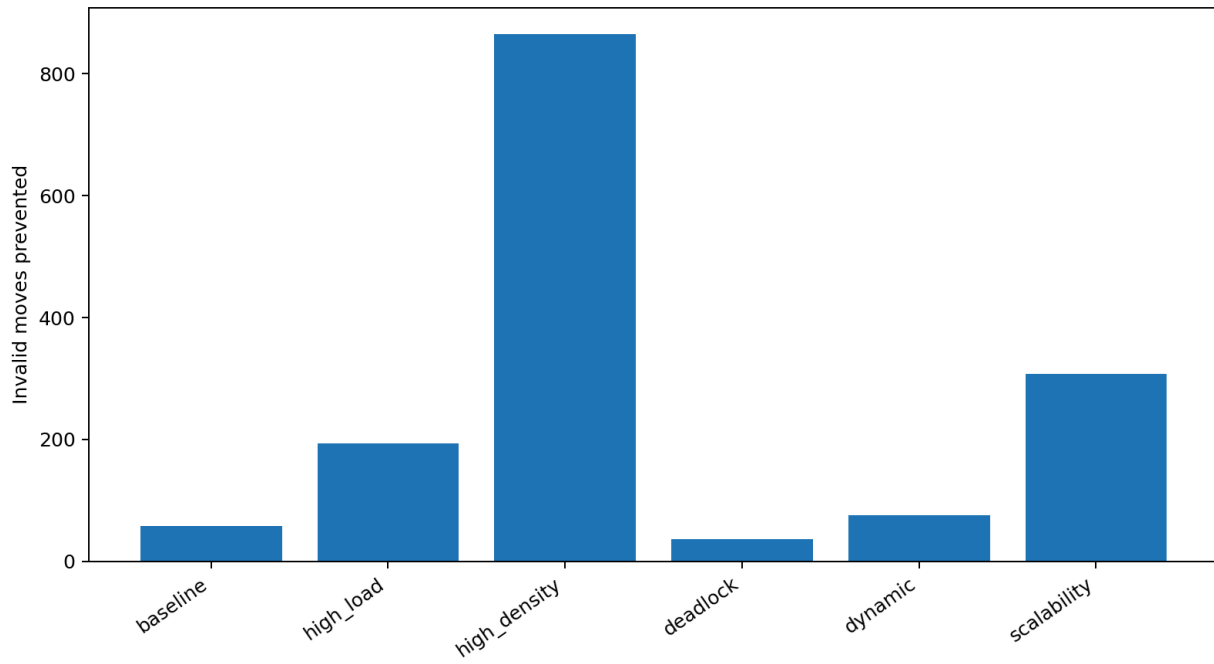


Figure 5: Invalid moves prevented by scenario.

High-density produced 865 invalid move preventions, far above the other scenarios. This proves that the collision-prevention logic was heavily exercised. The important result is that despite this pressure, actual collision events remained zero.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

7.6 Deadlocks Detected

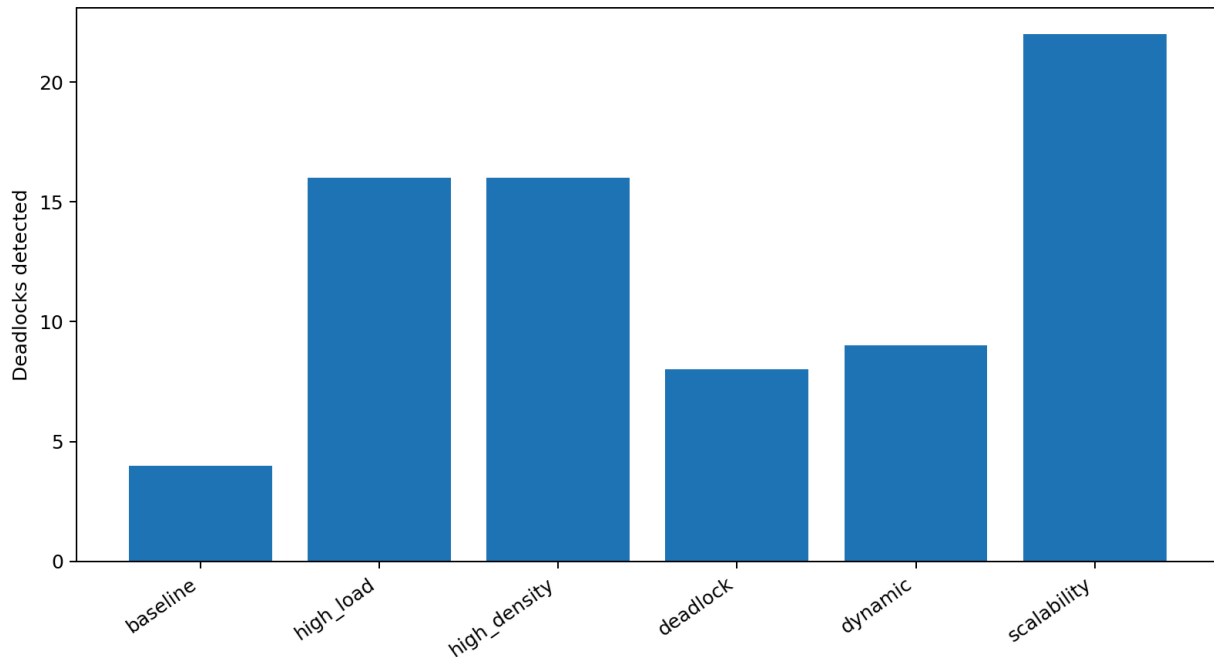


Figure 6: Deadlocks detected by scenario.

Deadlocks appeared more frequently in scalability and high-density scenarios. This is expected because more agents increase the probability of blocking cycles. The system detected and resolved these events without producing collisions.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

8. Stress Testing

8.1 High-Load Scenario

The high-load scenario increased the task generation rate from 0.18 to 0.45 and used 8 agents.

Metric	Value
Tasks created	128
Tasks completed	49
Throughput	0.163
Avg completion time	85.86
Collision events	0
Invalid moves prevented	194
Deadlocks detected	16

The higher throughput shows that the system processed more work under heavier demand. However, average completion time also increased sharply, meaning the system became more saturated.

This is normal: more tasks increase productivity but also increase queueing, waiting, and route contention.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

8.2 High-Density Scenario

The high-density scenario used 14 agents in the same warehouse layout.

Metric	Value
Tasks created	73
Tasks completed	57
Throughput	0.190
Avg completion time	47.61
Collision events	0
Invalid moves prevented	865
Deadlocks detected	16
Agent utilization	0.457

This scenario is the strongest congestion test. The system completed more tasks than baseline and still had zero collisions. However, utilization dropped sharply and invalid move prevention increased dramatically.

This means the collision-prevention and reservation systems worked, but the warehouse became congested enough that many agents were blocked or forced to wait.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

8.3 Deadlock Scenario

The deadlock scenario used a narrow-corridor map intended to provoke blocking.

Metric	Value
Tasks created	99
Tasks completed	42
Throughput	0.140
Avg completion time	90.69
Collision events	0
Deadlocks detected	8
Agent utilization	0.912

The system maintained zero collisions despite the deliberately constrained layout. The high average completion time confirms that the environment was difficult, but deadlock recovery allowed the simulation to continue.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

8.4 Long High-Load Run: 6000 Timesteps

A long high-load run was executed for 6000 timesteps to test extended stability.

Metric	Value
Total time	6000
Tasks created	2826
Tasks completed	1172
Throughput	0.195
Avg completion time	1777.79
Collision events	0
Invalid moves prevented	2788
Deadlocks detected	321
Agent utilization	0.878
Social welfare	10002.55
Average utility	1250.32

This stress run confirms long-run safety: even after 6000 timesteps and thousands of generated tasks, collision events remained zero.

However, the very high average completion time shows that task generation exceeded the system's long-term clearing capacity. In other words, the agents remained safe and productive, but the queue grew under sustained high load.

This is an important limitation: the strategy is robust, but high task generation rates require either more warehouse capacity, better task throttling, more drop-off/pickup points, or larger grid layouts.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

9. Discussion of Results

9.1 Why Throughput Increased Under Load

High-load, dynamic, high-density, and scalability scenarios generally completed more tasks per timestep than baseline because agents spent less time idle. More tasks were available, so the auction mechanism had more opportunities to assign work.

However, higher throughput does not automatically mean better system health. High-load also increased average completion time and deadlocks. High-density increased invalid move prevention dramatically. Therefore, throughput must be interpreted together with completion time, waiting, deadlocks, and utility.

9.2 Why High Density Reduced Average Utility

High-density completed 57 tasks, more than baseline. But the average utility per agent fell to 19.55.

The reason is congestion. Fourteen agents in the same grid create more local conflicts, more blocked cells, more replanning, and more waiting. The result is a system that completes more total work but is less efficient per agent.

This confirms a core multi-agent systems principle: increasing the number of agents improves parallelism only up to the point where coordination costs dominate

9.3 Why Collision Events Stayed at Zero

Collision events remained zero across all tested scenarios.

This happened because unsafe moves were prevented before execution using:

1. Occupancy checks
2. Vertex conflict detection
3. Edge-swap conflict detection
4. Reservation-table constraints
5. Final safety gate before synchronous movement execution

The high-density scenario is especially strong evidence because it had 865 invalid moves prevented but still zero actual collisions. This means the simulator did not avoid collisions by luck; it actively blocked unsafe moves.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

9.4 Why Deadlocks Appeared but Did Not Break the Simulation

Deadlocks appeared in stress scenarios because agents shared narrow corridors, pickup/drop-off regions, and reservation windows.

The important result is not that deadlocks never happened. In dense multi-agent systems, some blocking cycles are expected.

The important result is that the system detected persistent blocking and recovered without collisions.

Deadlock detection and recovery therefore acted as a resilience mechanism rather than a guarantee that blocking never occurs.

9.5 Best Overall Scenario

The scalability scenario produced the highest throughput and total social welfare:

throughput = 0.257

tasks_completed = 77

social_welfare = 620.60

This suggests that increasing the number of agents can improve system-level productivity when the environment is not too constrained.

The dynamic scenario was the most balanced configuration:

throughput = 0.187

avg_completion_time = 59.12

social_welfare = 506.70

average_utility = 63.34

It improved throughput over baseline without the severe utility collapse observed in high-density.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

10. Threats to Validity and Limitations

1. Simulation-only validation
The system is a discrete-time Python simulation, not a physical robot controller.
2. Centralized simulation loop
Agents use decentralized-style decision logic, but the simulation executes inside a centralized synchronous loop for reproducibility.
3. Fixed warehouse layouts
Layouts are deterministic. This supports repeatability but does not cover every possible warehouse geometry.
4. Utility function is derived
Utility and social welfare are computed from exported metrics rather than directly optimized by the agents.
5. Naive baseline comparison scope
A naive baseline controller was implemented and compared numerically against the advanced strategy on the baseline scenario. The naive controller uses nearest-agent assignment and standard A* without cooperative reservations, edge-swap prevention, congestion-aware bidding, or deadlock recovery. A future extension could run the naive-vs-advanced comparison across all stress scenarios, not only the baseline scenario.
6. Long high-load saturation
The 6000-step high-load run showed that the system remains safe, but the task queue grows when demand exceeds clearing capacity.

10.1 Mapping Results to SRS/SDD Requirements

The validation results support the main SRS and SDD goals of the project.

Dynamic task generation was validated through the created and completed task counts in each scenario. The baseline scenario generated 58 tasks and completed 42 tasks, while the scalability scenario generated 96 tasks and completed 77 tasks. This confirms that the simulator supports continuous task creation and task lifecycle tracking.

Auction synchronization was validated through deterministic task assignment. Each task is assigned to one winning agent using the lowest bid, with the lowest agent ID used as a deterministic tie breaker. This prevents multiple agents from claiming the same task.

Collision avoidance was validated by the strongest safety result in the report: all tested scenarios produced zero actual collision events. This supports the SDD design choice of using a reservation table, vertex conflict prevention, edge-swap prevention, and a final safety check before movement execution.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

Cooperative path planning was validated by successful task execution under normal, high-load, high-density, dynamic, and scalability conditions. The agents were able to plan and replan paths while sharing the same warehouse grid.

Deadlock handling was validated through the deadlock-focused and stress scenarios. Deadlocks were detected in difficult layouts and dense conditions, but the simulation continued and maintained zero collisions. This confirms that deadlock detection and recovery acted as resilience mechanisms rather than merely documented design features.

Metrics and logging requirements were validated through exported JSON and CSV files, including summaries, event logs, agent decision traces, and timestep snapshots. These outputs provide evidence for task completion, throughput, utility, social welfare, invalid move prevention, deadlock detection, and agent utilization.

Overall, the validation results show that the implementation is aligned with the SRS and SDD at the simulation level. The main functional requirements were not only implemented, but also tested under multiple operating conditions.

Validation & Testing Report

Swarm Logistics – Warehouse Automation Simulation

11. Conclusion

The validation results show that the engineered multi-agent strategy achieves the main system goals.

Across all tested scenarios:

`collision_events = 0`

This is the strongest safety result. The system successfully prevented unsafe movement even in high-density and long high-load stress tests.

The auction mechanism assigned tasks deterministically, Cooperative A* generated coordinated paths, the reservation table prevented vertex and edge-swap conflicts, and deadlock recovery allowed the system to continue under blocking conditions.

The results also show realistic tradeoffs:

1. More tasks increase throughput but also increase queueing and completion time.
2. More agents improve parallelism but can reduce average utility under congestion.
3. Stress scenarios increase invalid move prevention and deadlocks, but the system remains safe.

The direct naive-vs-advanced comparison further supports this conclusion. The advanced strategy completed more tasks, eliminated collisions, reduced invalid move attempts, and achieved much higher social welfare than the naive baseline. This confirms that the coordination mechanisms are not just implementation details; they are the main reason the system remains safe and productive.

Final assessment: the implementation is valid as a simulation of a coordinated multi-agent warehouse system. It satisfies the validation goals by demonstrating task completion, collision avoidance, deterministic task allocation, stress handling, and measurable utility/social welfare behavior.